# Lesson 02 Demo 01

# Building an RAG-Powered FAQ Agent with Custom Knowledge

**Objective:** To build an RAG-powered FAQ agent using LangChain, FAISS, and an LLM that retrieves relevant document chunks and generates grounded, source-backed answers to solve ungrounded or inconsistent responses and speed up accurate support

**Tools required:** Python, LangChain, FAISS, Azure OpenAI API, and Visual Studio Code

**Prerequisites:** Familiarity with LangChain and RAG concepts

**Steps to be followed:**

1. Set up your local environment in Visual Studio Code
2. Load and chunk your custom FAQ document
3. Configure embeddings and create the vector store
4. Initialize the LLM and set up the RAG chain
5. Query the RAG agent and view results

## Step 1: Set up your local environment in Visual Studio Code

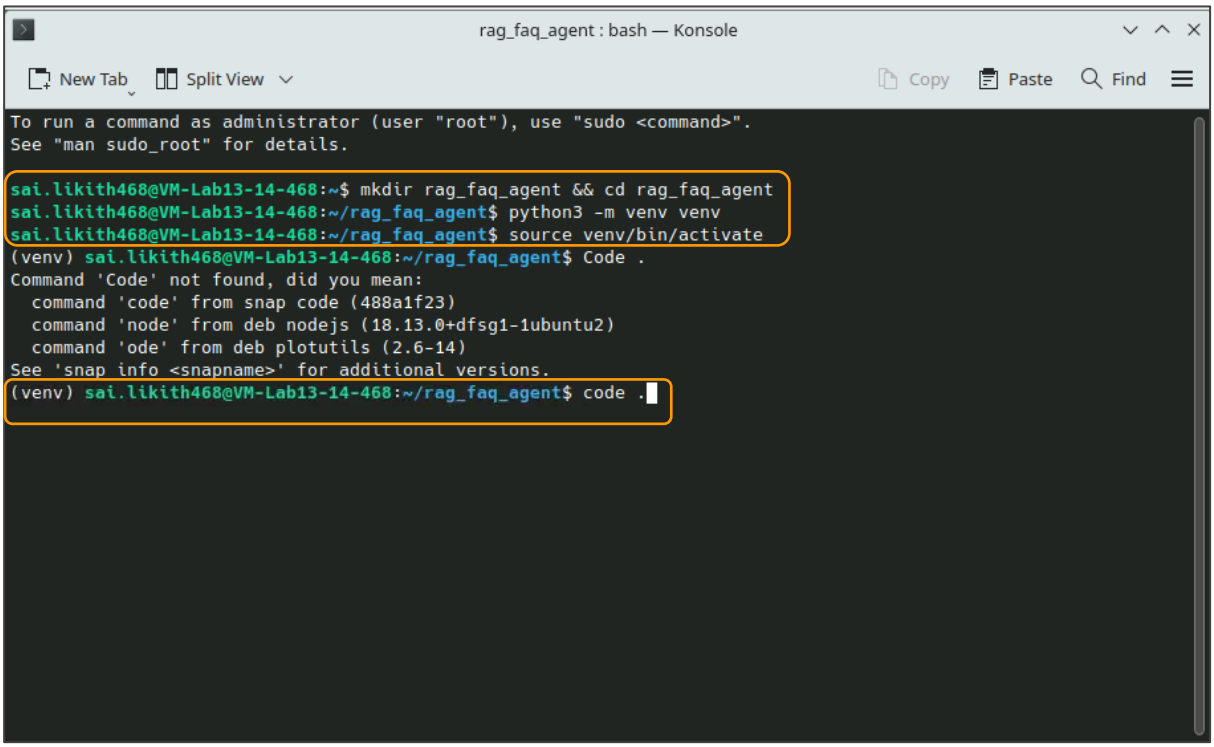
1.1 Open the terminal on your system and run the following commands:

**mkdir rag_faq_agent && cd rag_faq_agent**

**python3 -m venv venv**

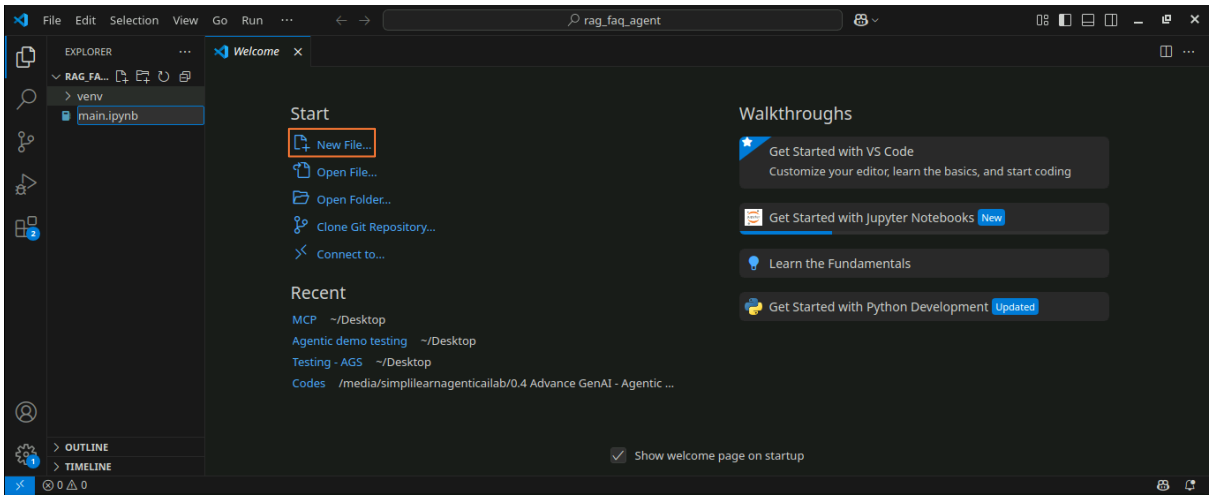
**source venv/bin/activate**

**code .**



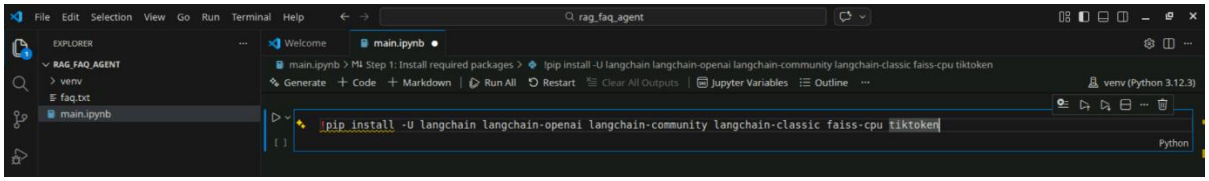

**Note:** This creates a virtual environment inside the rag_faq_agent directory and opens the folder in Visual Studio Code.

1.2 Click **New File...** to create a new file, and name it **main.ipynb**

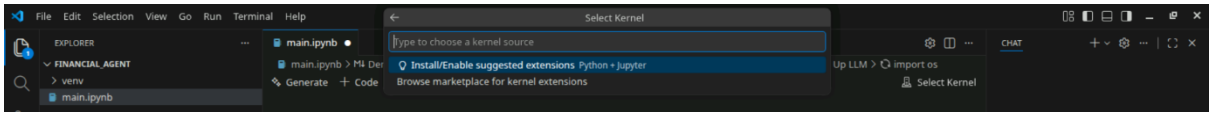



1.3 In the first cell in the file, run the following command to install required packages:

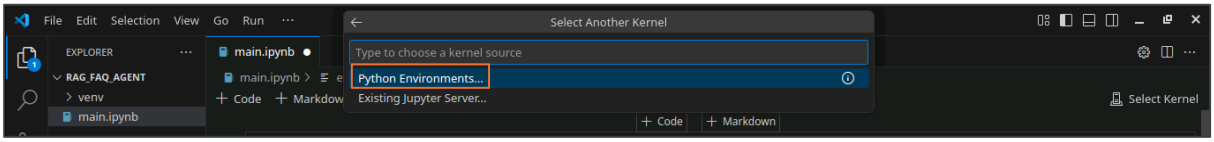
**!pip install -U langchain langchain-openai langchain-community langchain-classic faiss-cpu tiktoken**

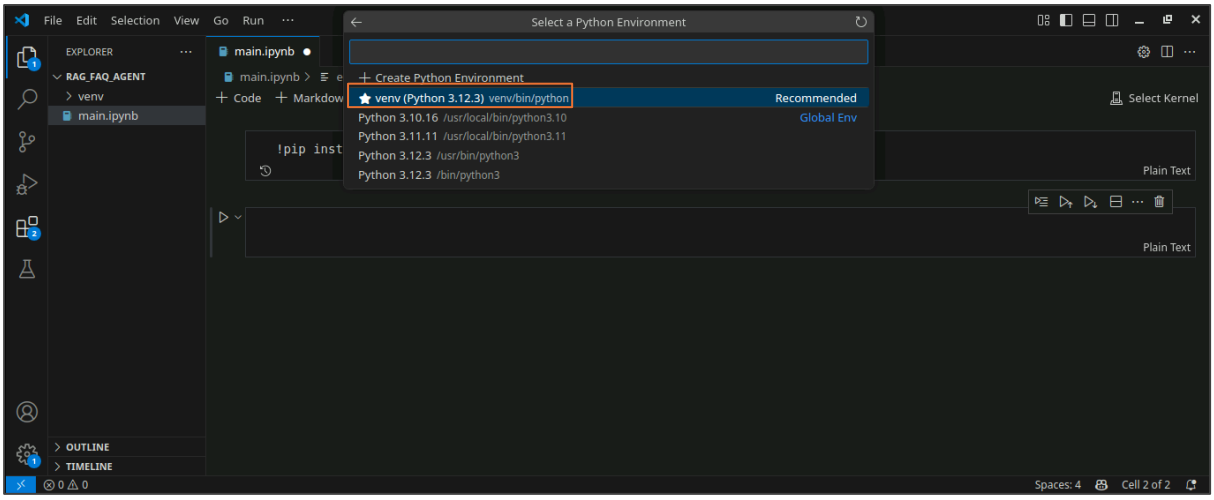



1.4 When prompted, click **Install/Enable suggested extensions Python + Jupyter**.

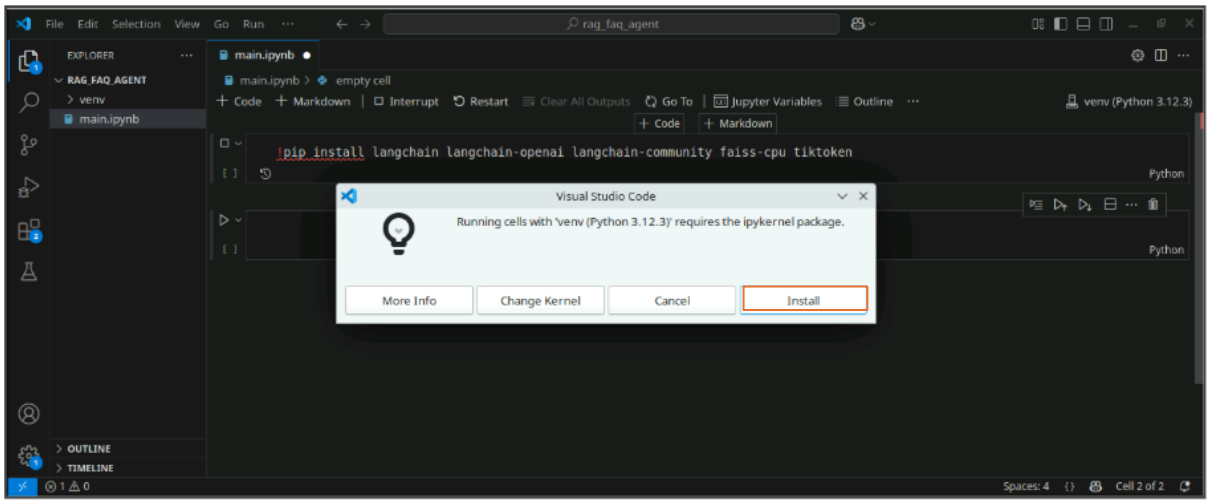



1.5 When prompted, select **Python Environments...** and then choose the venv interpreter **venv (Python 3.12.3)**

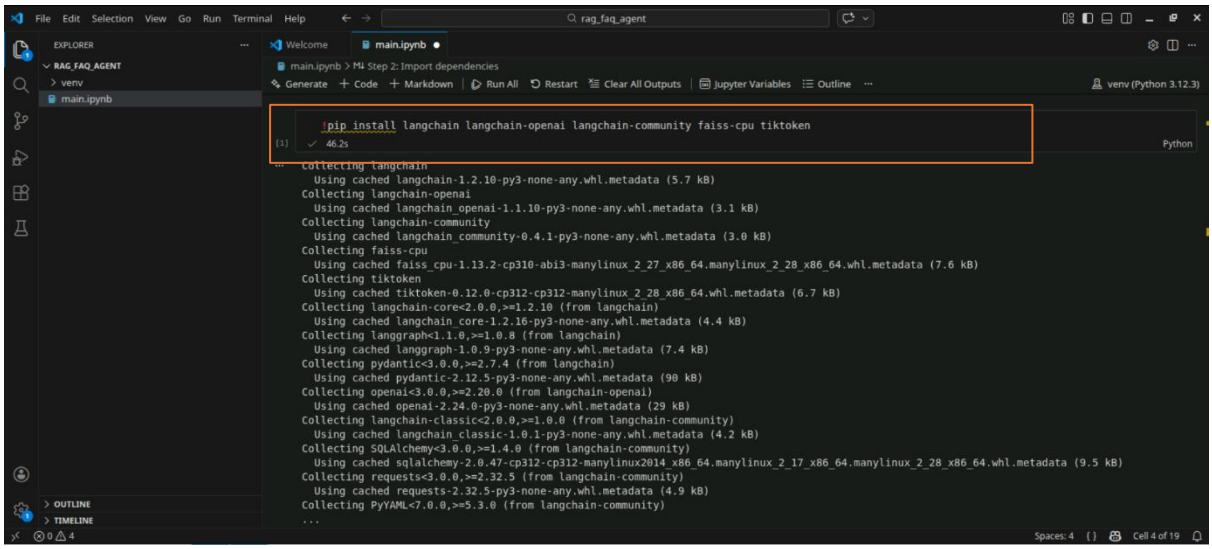





1.6 If prompted to install ipykernel, click **Install**

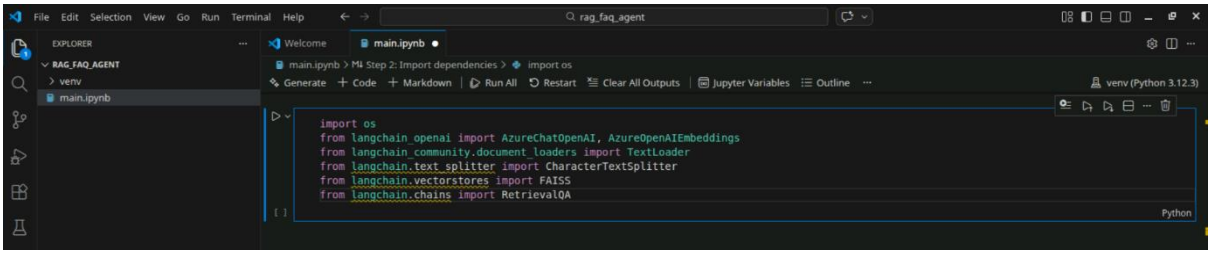



1.7 Rerun the installation cell to complete the package setup




1.8 Import the required libraries using the following code:

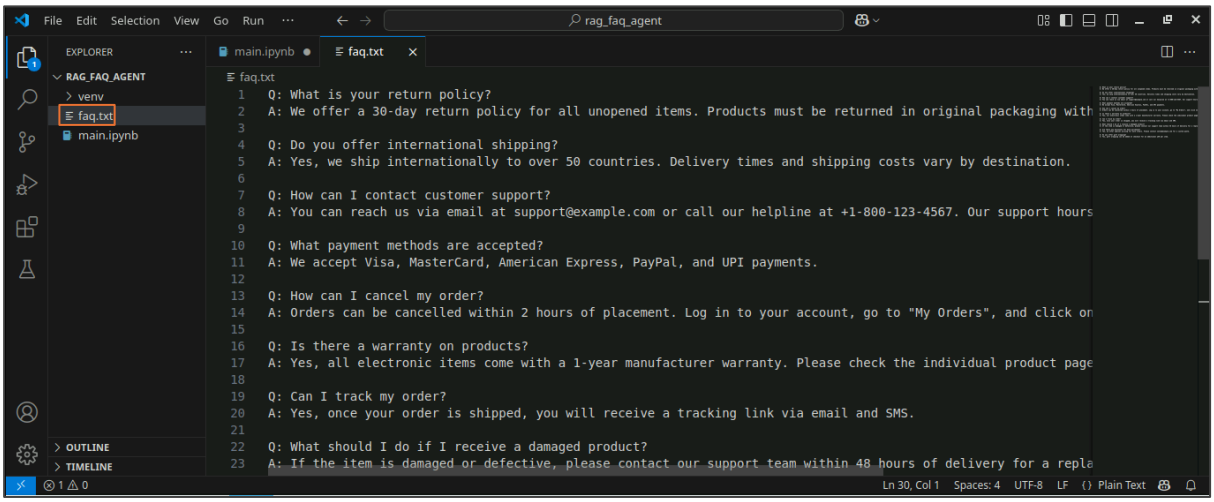
```
import os
from langchain_openai import AzureChatOpenAI, AzureOpenAIEmbeddings
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_classic.chains import RetrievalQA
```




> **Note:** These imports load classes for document loading, text chunking, vector storage, LLM access, and QA pipelines using LangChain and Azure OpenAI.

## Step 2: Load and chunk your custom FAQ document

2.1 Create a new file named **faq.txt** in the same directory and copy and paste the content from the resource file provided. Press **Ctrl** + **S** to save the file.



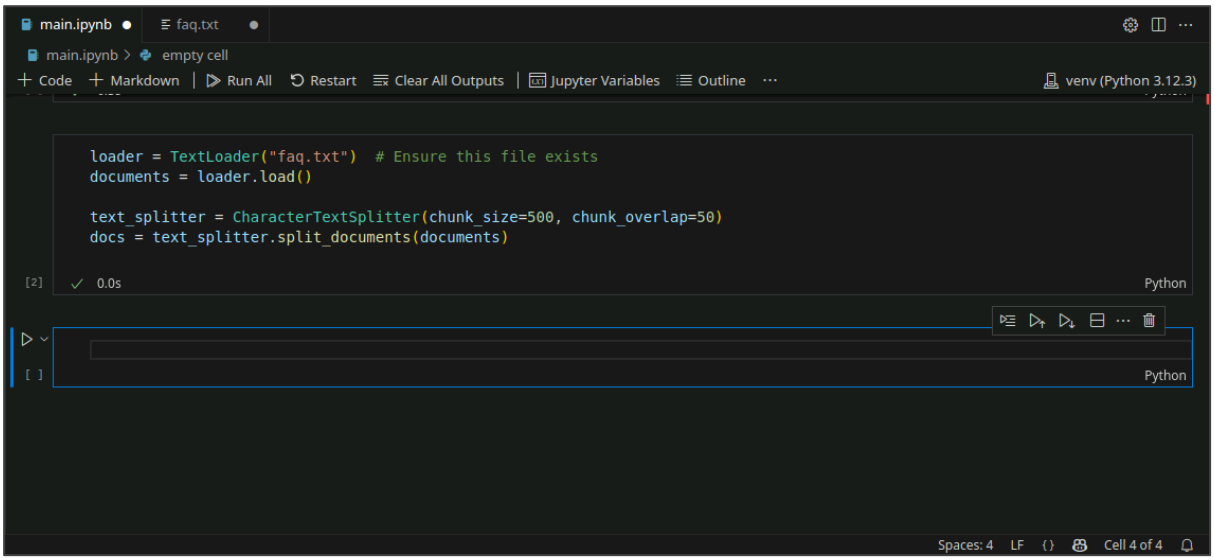

> Note: You can download the resource file (faq.txt) from the Reference Materials section.

2.2 Load and split the document using the following code in the main.ipynb file:

```
loader = TextLoader("faq.txt")  # Ensure this file exists
documents = loader.load()

text_splitter = CharacterTextSplitter(chunk_size=500, chunk_overlap=50)
```

**docs = text_splitter.split_documents(documents)**

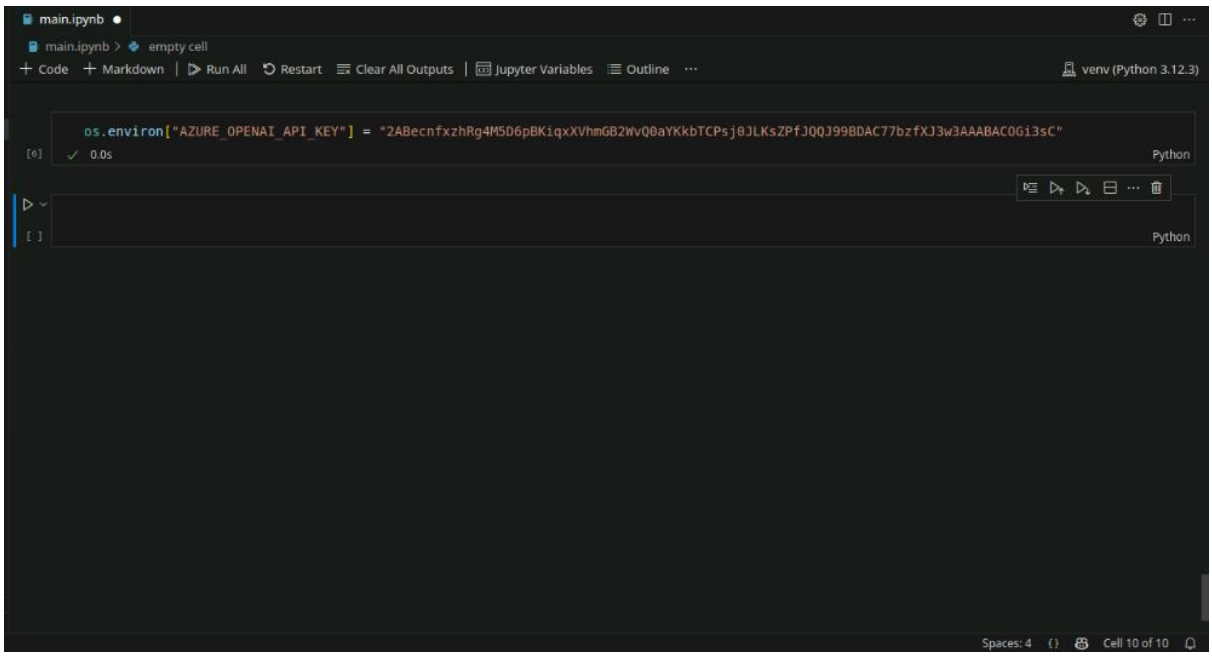



**Note:** The document is split into smaller chunks to enable semantic search during retrieval.

## Step 3: Configure embeddings and create the vector store

3.1 Set your Azure OpenAI API key using the following command:

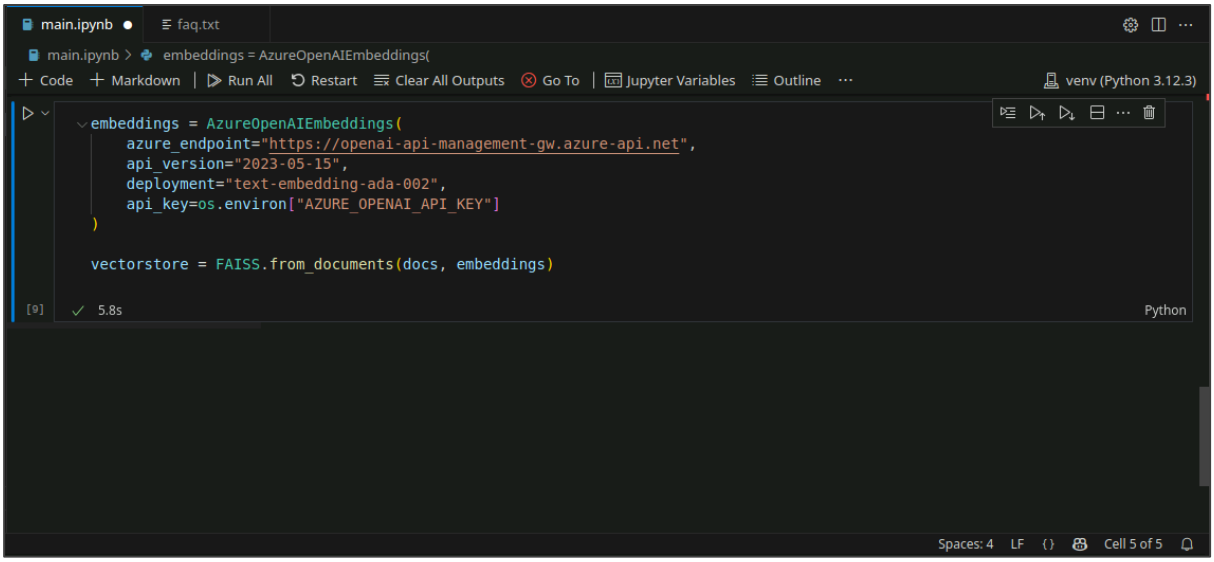
**os.environ["AZURE_OPENAI_API_KEY"] = "2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77bzfXJ3w3AAABACOGi3sC"**



3.2 Generate embeddings and store them in FAISS using the following code:

```
embeddings = AzureOpenAIEmbeddings(
    azure_endpoint="https://openai-api-management-gw.azure-api.net",
    api_version="2023-05-15",
    deployment="text-embedding-ada-002",
    api_key=os.environ["AZURE_OPENAI_API_KEY"]
)

vectorstore = FAISS.from_documents(docs, embeddings)
```

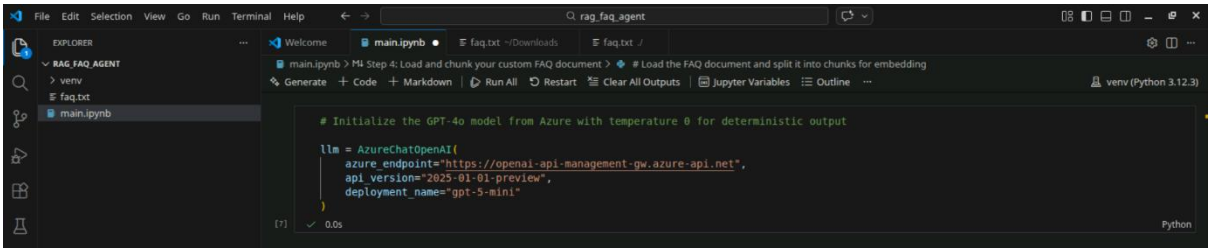



**Note:** The embedding model text-embedding-ada-002 transforms chunks into vectors, which are then stored in FAISS for similarity-based retrieval. FAISS (Facebook AI Similarity Search) is an open-source library for fast vector search.

## Step 4: Initialize the LLM and set up the RAG chain

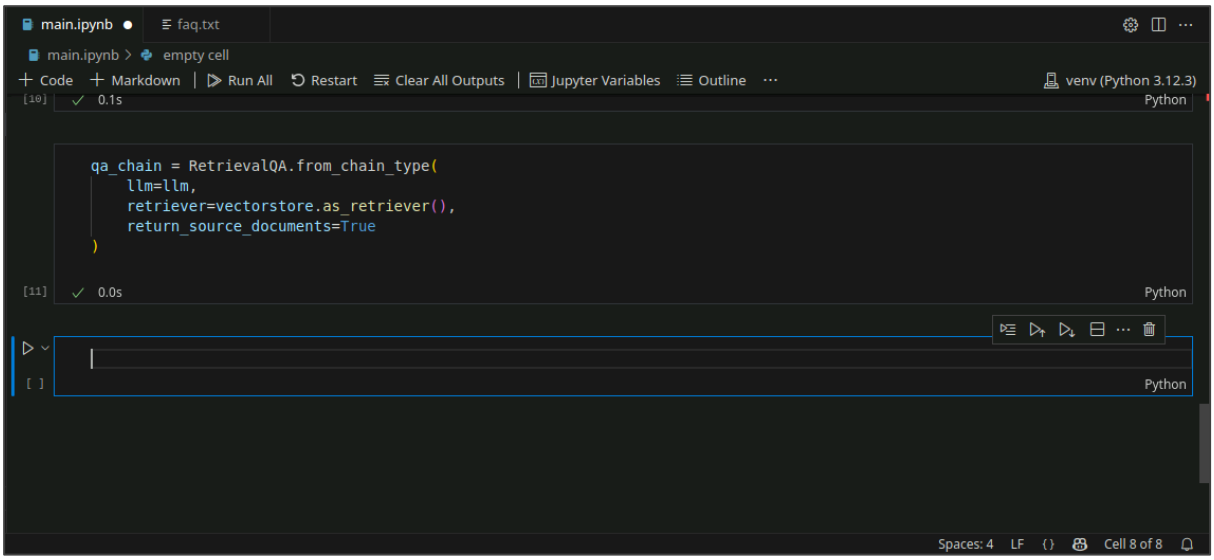
4.1 Create a GPT-5 LLM instance using the following code:

```
llm = AzureChatOpenAI(
    azure_endpoint="https://openai-api-management-gw.azure-api.net",
    api_version="2025-01-01-preview",
    deployment_name="gpt-5-mini"
)
```




4.2 Initialize the RAG pipeline using the following code:

```
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectorstore.as_retriever(),
    return_source_documents=True
)
```

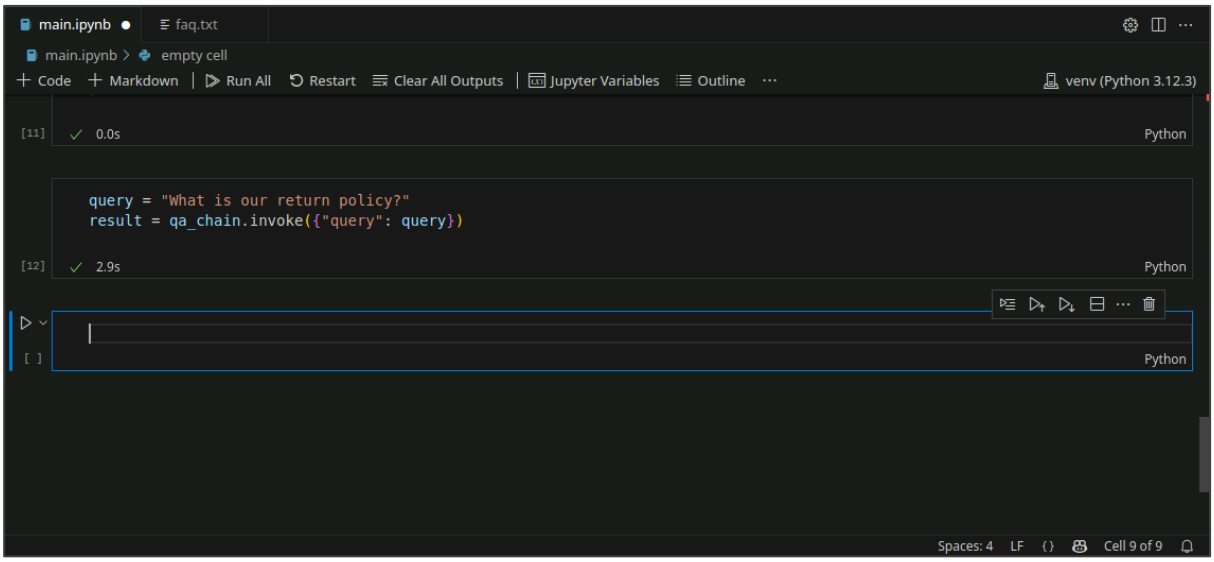



**Note:** This sets up a LangChain pipeline that combines retrieval with GPT-based reasoning and provides source context along with the answer.

## Step 5: Query the RAG agent and view results

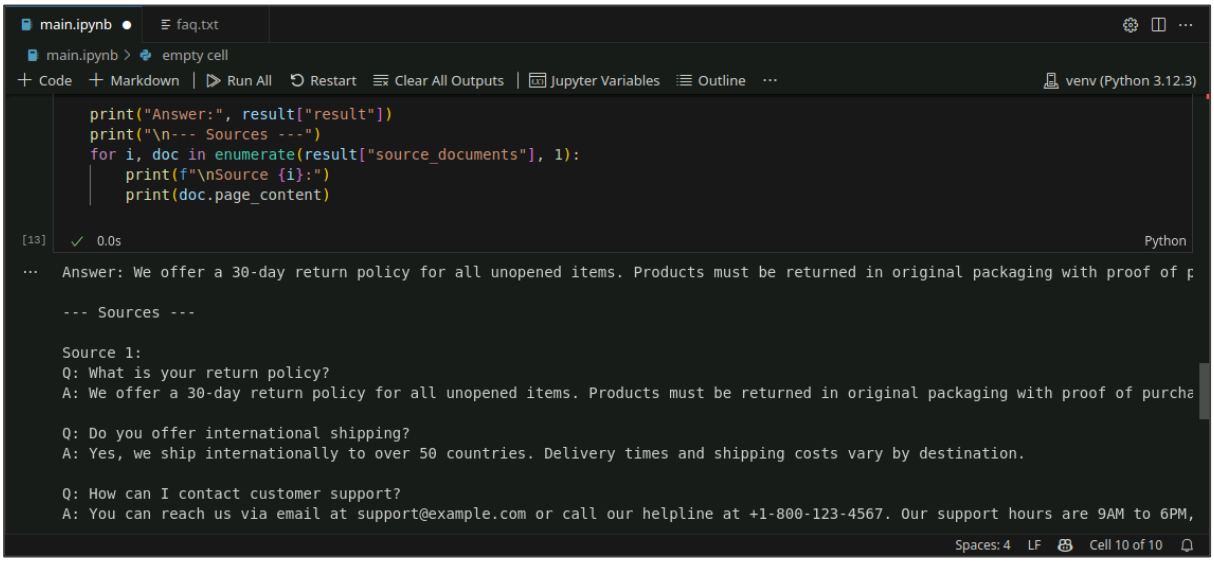
5.1 Ask a sample question within the query parameter:

```
query = "What is our return policy?"
result = qa_chain.invoke({"query": query})
```




5.2 Display the final answer and its source context using the following code:

```
print("Answer:", result["result"])
print("\n--- Sources ---")
for i, doc in enumerate(result["source_documents"], 1):
    print(f"\nSource {i}:")
    print(doc.page_content)
```



**Note:** The output includes both the generated answer and the specific document snippets used to produce it.

By following these steps, you have successfully set up a complete RAG workflow that indexes your FAQ content, retrieves the best matches, and returns cited answers, reducing hallucinations, improving answer accuracy, and centralizing domain knowledge.